



/dev/world
2019

Updating Your App for iOS 13

Zach Simone



In Partnership With



Updating Your App for iOS 13

Zach Simone - [/dev/world/2019](#)

Hi, I'm Zach 🖐️

- Final-year student @ University of Technology, Sydney
- iOS/tvOS developer @ Kayo Sports
- Co-organiser CocoaHeads Sydney
- Co-host *Cup of Tech* podcast



Thanks to the AUC for sponsoring our trip to WWDC! 😊

So, how about that WWDC?



Shiny new stuff - yay!

- SwiftUI
- Catalyst
- AR/ML
- Shortcuts improvements
- Sign In with Apple
- iPadOS
- iOS dark mode

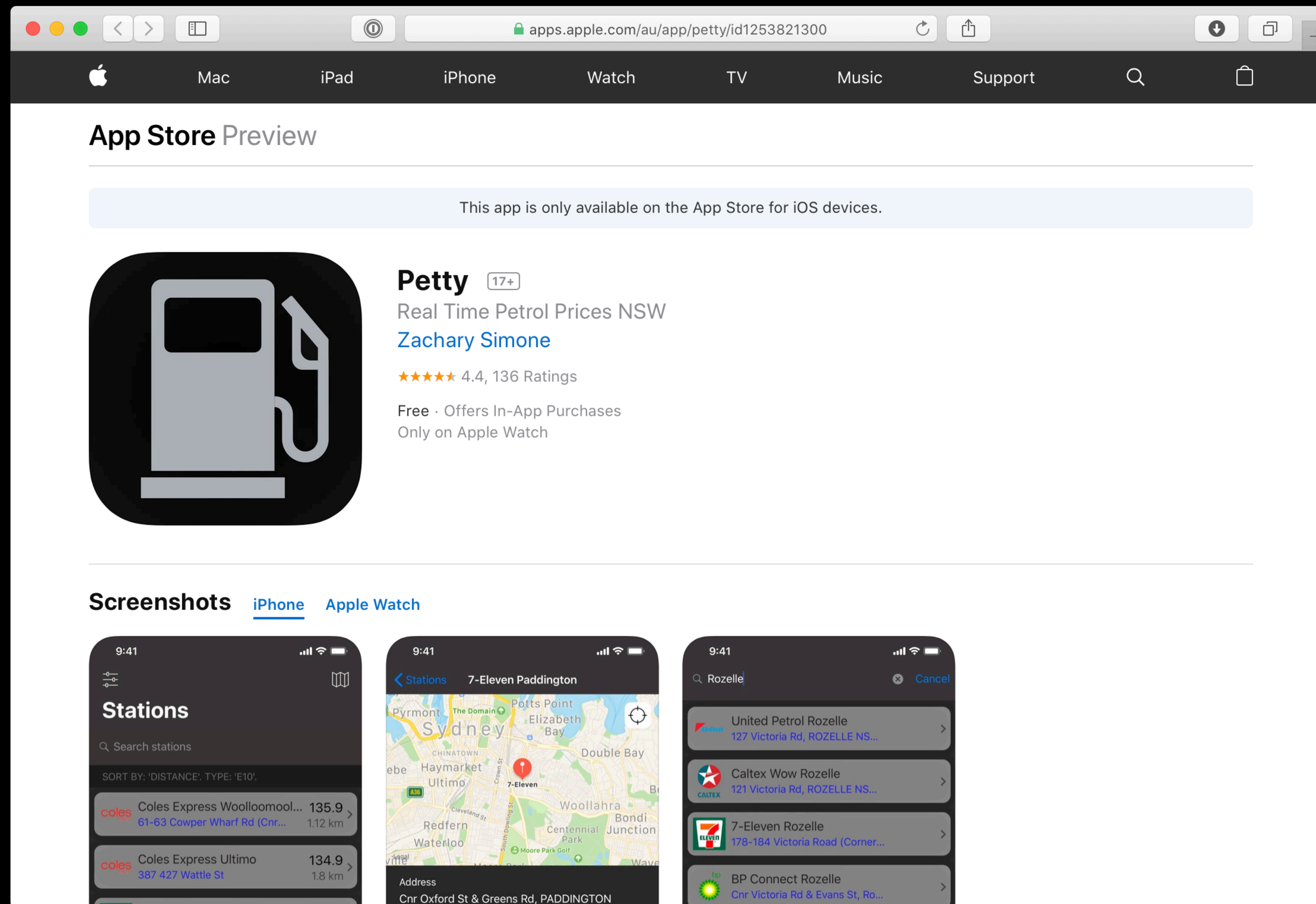
Some less-shiny but cool stuff

- System colours
- SF Symbols
- Independent watch apps
- New modal presentation styles

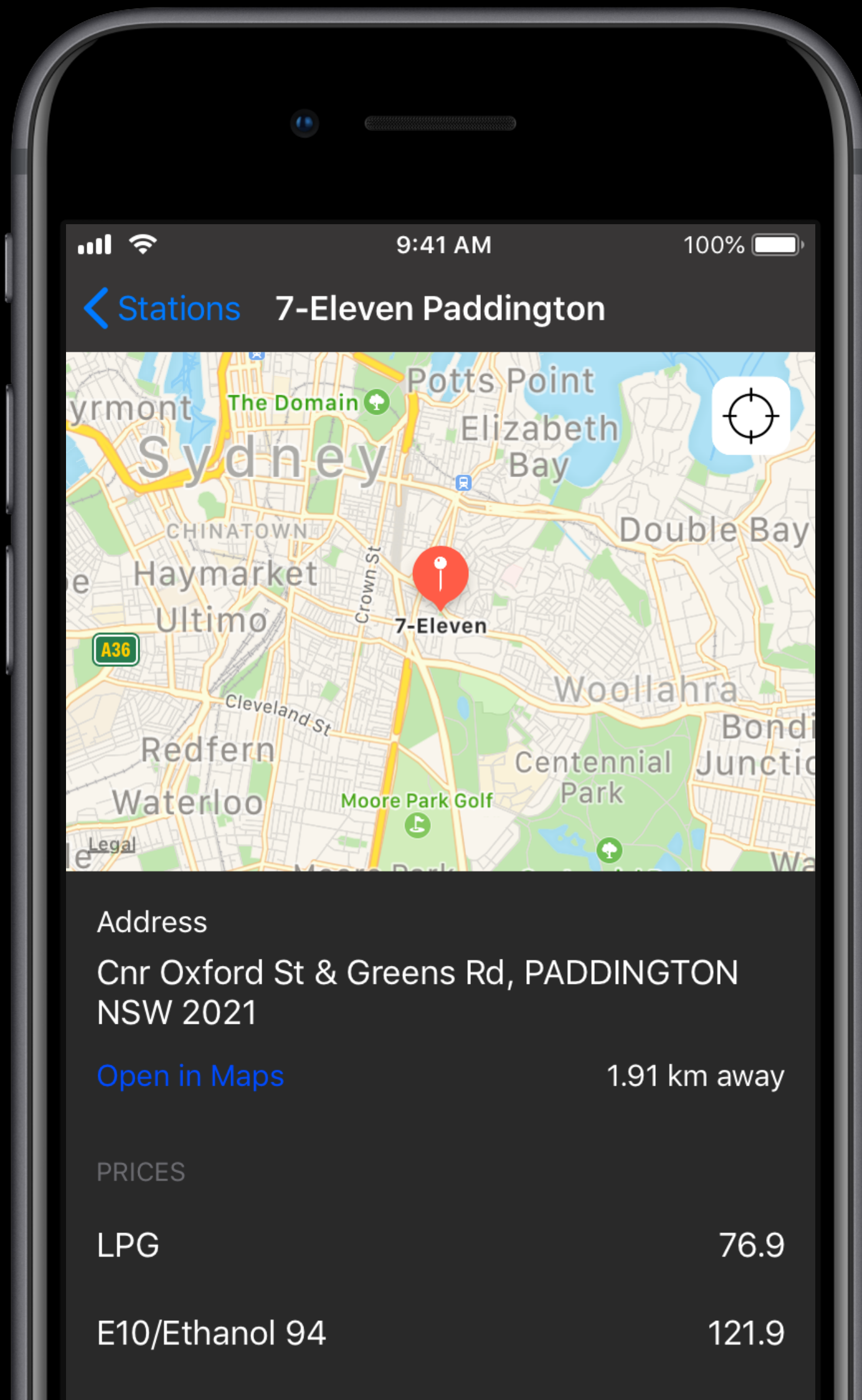
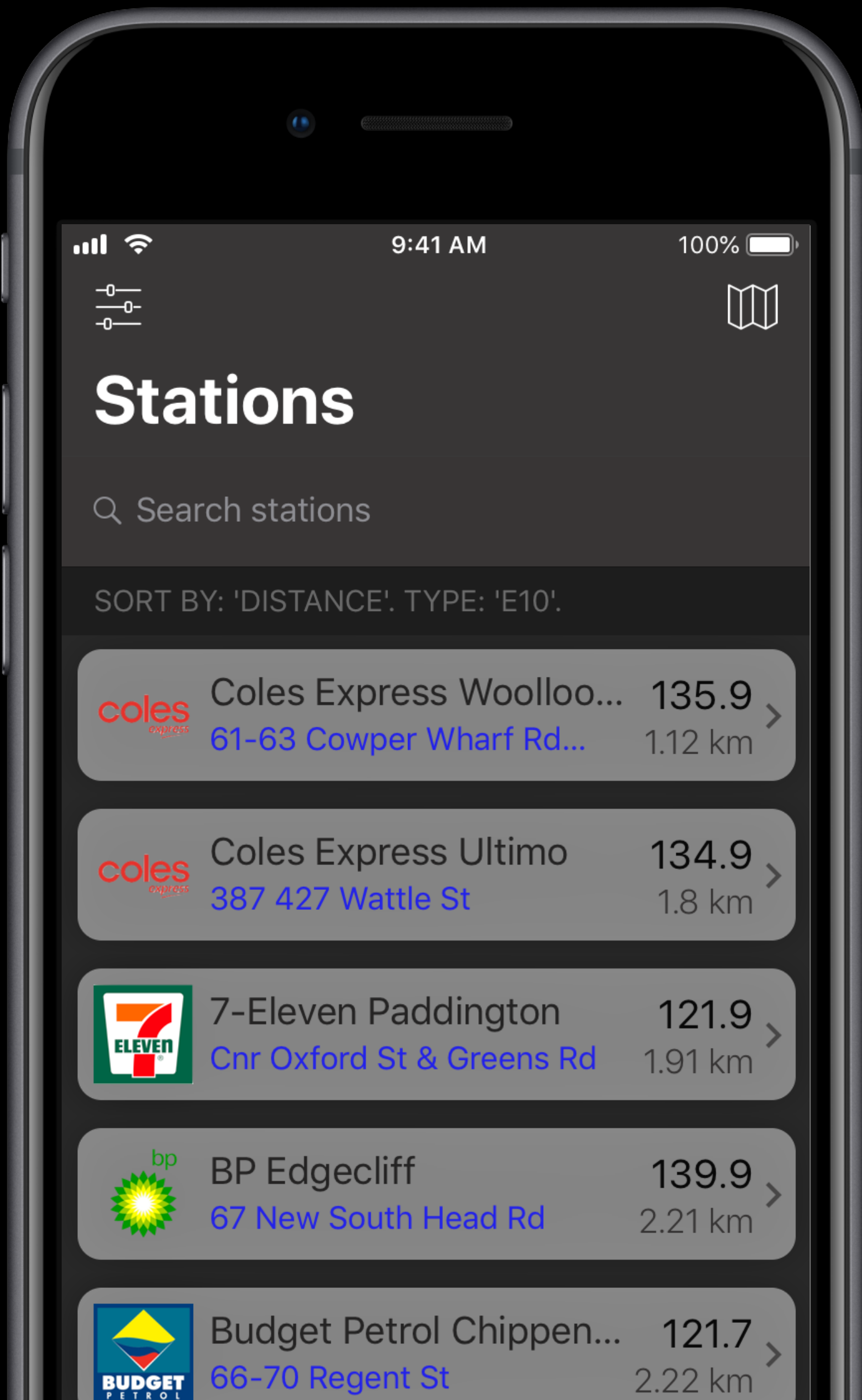
Where to begin?



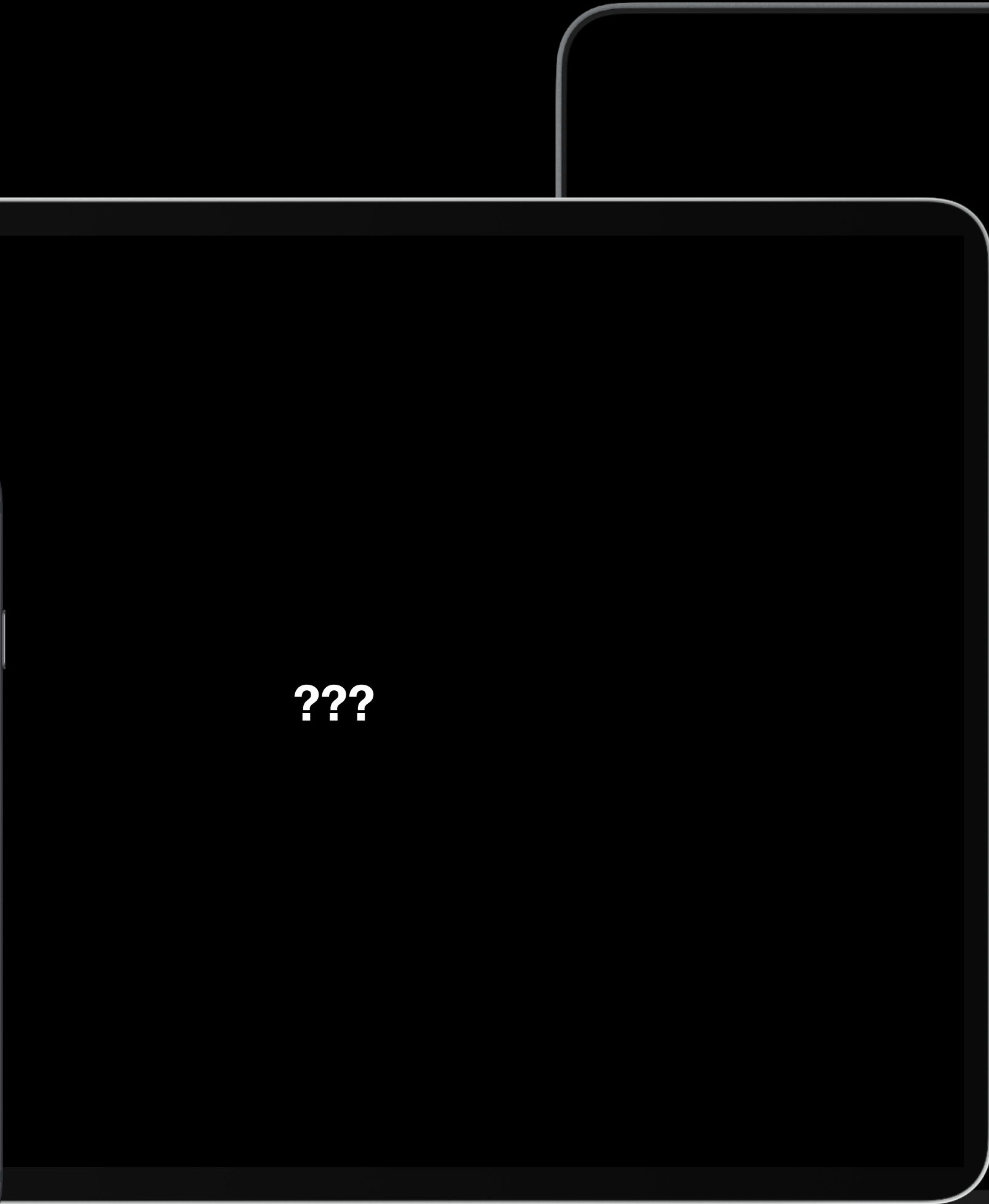
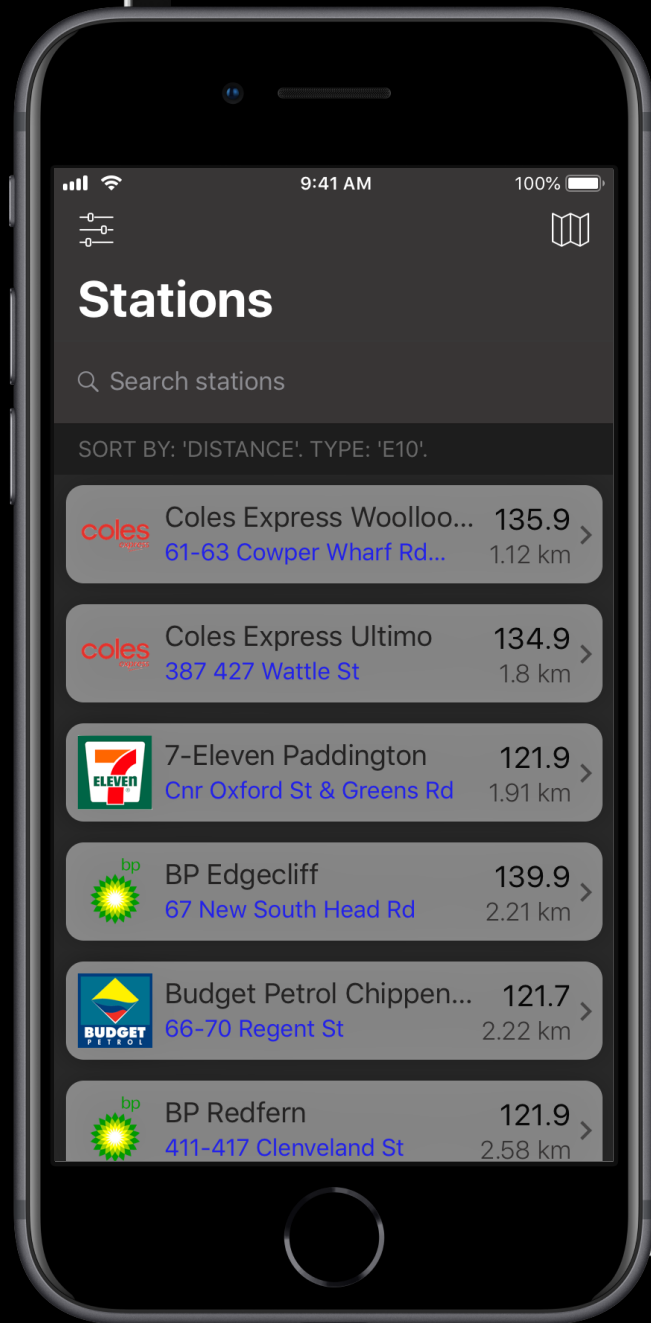
First, you need an app



Petty - Real-Time Petrol Prices*



* Only in NSW, Australia's best state



Updating for watchOS 6



Independent watch apps

- Allow an app to run independently of the phone
- The app bundle can be downloaded to the watch separately to the iOS app bundle
- No guarantee that the watch app can "phone home" using WatchConnectivity
- Possible to create a watchOS app without an iOS app attached whatsoever
- Better user experience to run the app independently

Petty grows up!

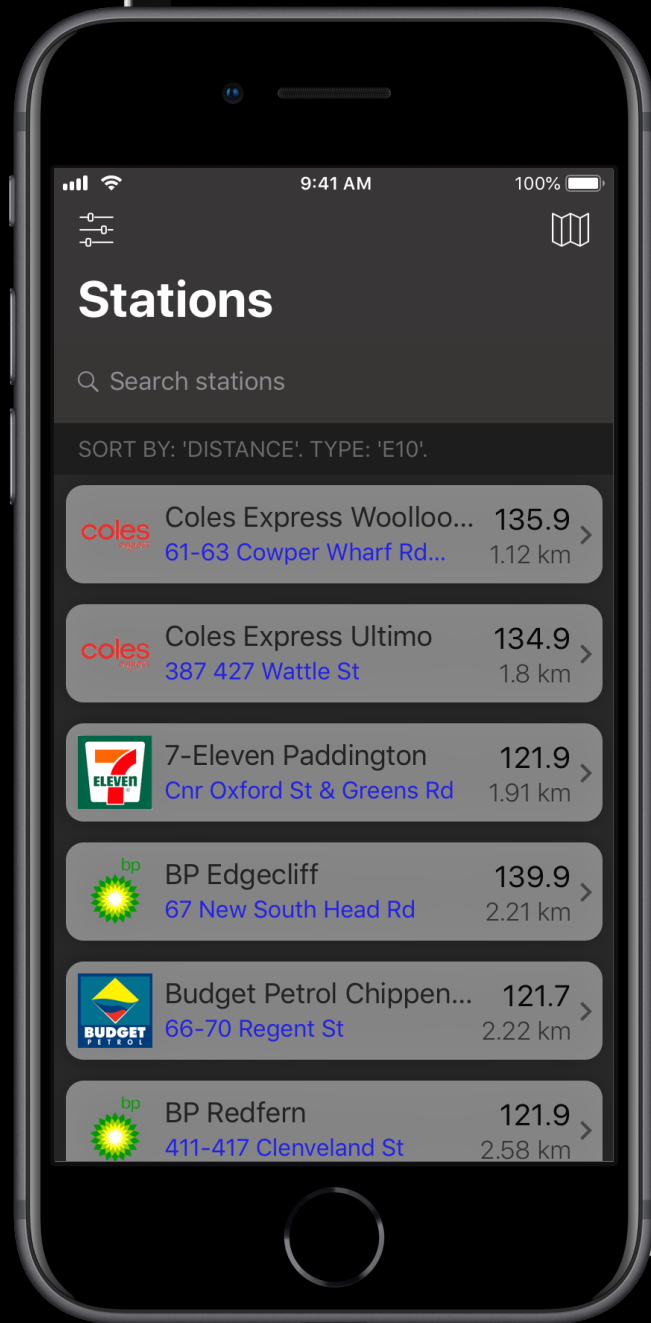
▼ Deployment Info

Deployment Target ▼

✓ Supports Running Without iOS App Installation

Possible for independent watch apps

- Sign-in and/or account creation.
 - Text Field API's (WKInterfaceTextField)
 - Sync with CloudKit
 - Sign in with Apple
- Permission requests including HealthKit
- Can send remote push notifications directly to the Apple Watch
- Can't rely on WatchConnectivity

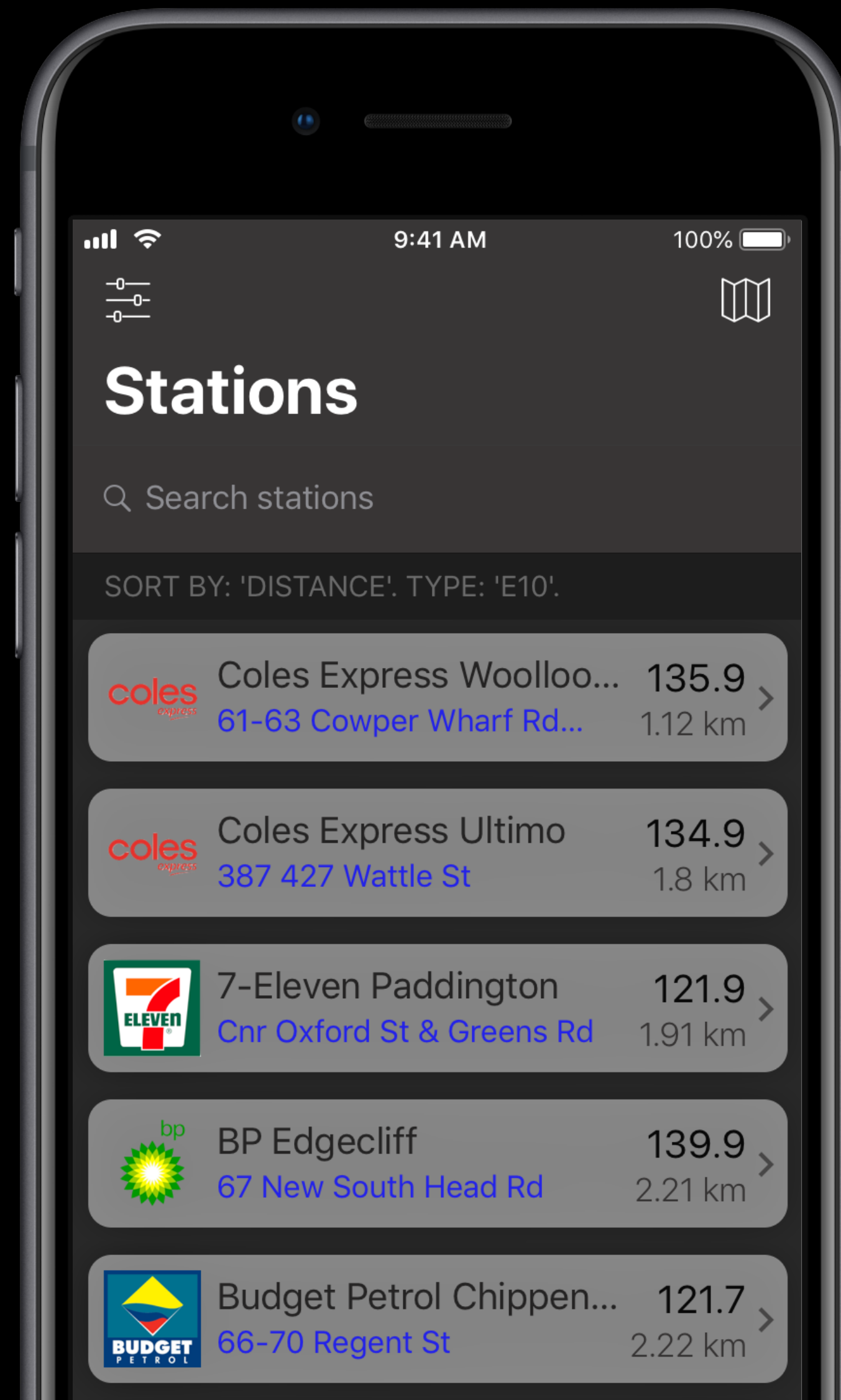


???

???

MacBook Pro

Updating for iOS 13



Scene Delegate

- Can now have multiple instances of your app's user interface on iOS
- Aids with support for multi-window on iPad, and macOS
- A `SceneDelegate` manages this (as opposed to the `AppDelegate`)
- One independent "scene" per window of your app
- Still use AppDelegate for non-window specific tasks - such as registering for push notifications

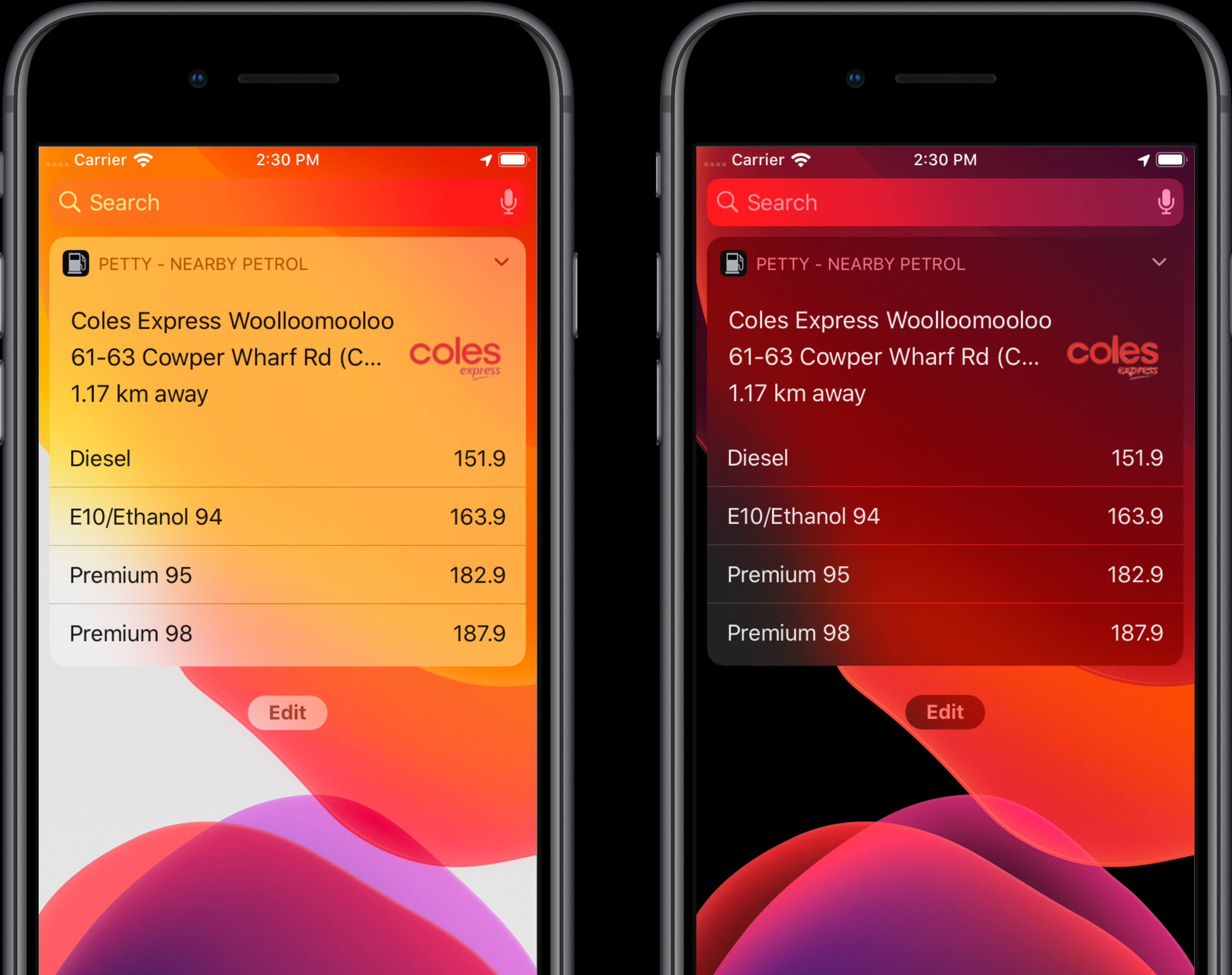


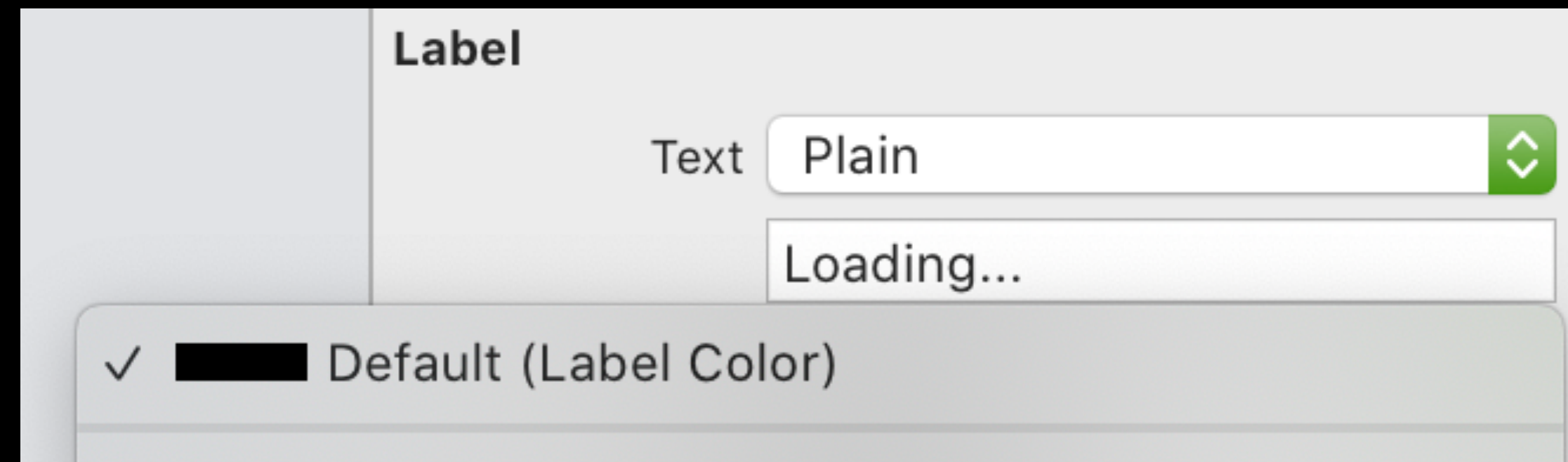
AppDelegate.swift



SceneDelegate.swift

Dark Mode





Use system colours!

Locking to dark (or light) mode

Info.plist (locks whole app)

```
<key>UIUserInterfaceStyle</key>  
<string>Dark</string>
```

Per UIViewController (or UIView) basis

```
overrideUserInterfaceStyle = .dark
```

But... more UI changes

- System colours (but not a total theme change)
 - Backgrounds
 - Labels
- SF Symbols
- Modal presentation styles

System colours

Human Interface Guidelines

OverviewResourcesVideosWhat's New

> iOS

> App Architecture

> User Interaction

> System Capabilities

> Visual Design

Adaptivity and Layout

Animation

Branding

Color

Dark Mode

Materials

Terminology

Typography

Video

> Icons and Images

> Bars

> Views

> Controls

> Extensions

macOS

tvOS

watchOS

> Technologies

System Colors

iOS offers a range of system colors that automatically adapt to vibrancy and changes in accessibility settings like Increase Contrast and Reduce Transparency. The system colors look great individually and in combination, on both light and dark backgrounds, and in both light and dark modes.

Don't hard-code system color values in your app.

The color values provided below are intended for reference during your app design process. The actual color values may fluctuate from release to release, based on a variety of environmental variables. Always use the API to apply system colors; for developer guidance, see [UIColor](#).

Default

Accessible

Light	Dark	Name	API
R 0 G 122 B 255	R 10 G 132 B 255	Blue	systemBlue
R 142 G 142 B 147	R 152 G 152 B 157	Gray	systemGray
R 52 G 199 B 89	R 48 G 209 B 88	Green	systemGreen
R 88 G 86 B 214	R 94 G 92 B 230	Indigo	systemIndigo

delines

OverviewResourcesVideosWhat's New

System Colors

iOS offers a range of system colors that automatically adapt to vibrancy and changes in accessibility settings like Increase Contrast and Reduce Transparency. The system colors look great individually and in combination, on both light and dark backgrounds, and in both light and dark modes.

Don't hard-code system color values in your app.

The color values provided below are intended for reference during your app design process. The actual color values may fluctuate from release to release, based on a variety of environmental variables. Always use the API to apply system colors; for developer guidance, see [UIColor](#).

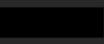
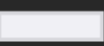
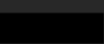
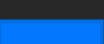
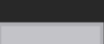
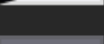
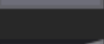
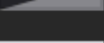
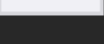
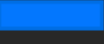
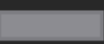
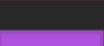
Default

Accessible

Light	Dark	Name	API
R 0 G 64 B 221	R 64 G 156 B 255	Blue	systemBlue
R 105 G 105 B 110	R 152 G 152 B 157	Gray	systemGray
R 36 G 138 B 61	R 48 G 219 B 91	Green	systemGreen

Dynamic system colours

Color	Used for	API
Label	A text label that contains primary content.	label
Secondary label	A text label that contains secondary content.	secondaryLabel
Tertiary label	A text label that contains tertiary content.	tertiaryLabel
Quaternary label	A text label that contains quaternary content.	quaternaryLabel
Placeholder text	Placeholder text in controls or text views.	placeholderText
Separator	A separator that allows some underlying content to be visible.	separator
Opaque separator	A separator that doesn't allow any underlying content to be visible.	opaqueSeparator
Link	Text that functions as a link.	link

Recently Used Colors	
	
	Dark Text Color
	Group Table View Background Color
	Label Color
	Light Text Color
	Link Color
	Opaque Separator Color
	Placeholder Text Color
	Quaternary Label Color
	Quaternary System Fill Color
	Scroll View Textured Background Color
	Secondary Label Color
	Secondary System Background Color
	Secondary System Fill Color
	Secondary System Grouped Background Color
	Separator Color
	System Background Color
	System Blue Color
	System Brown Color
	System Fill Color
	System Gray Color
	System Green Color
	System Grouped Background Color
	System Indigo Color
	System Orange Color
	System Pink Color
	System Purple Color
	System Red Color
	System Teal Color
	System Yellow Color
	Table Cell Grouped Background Color
	Tertiary Label Color
✓ 	Tertiary System Background Color
	Tertiary System Fill Color

Using system colours

SwiftUI

`Color.gray`

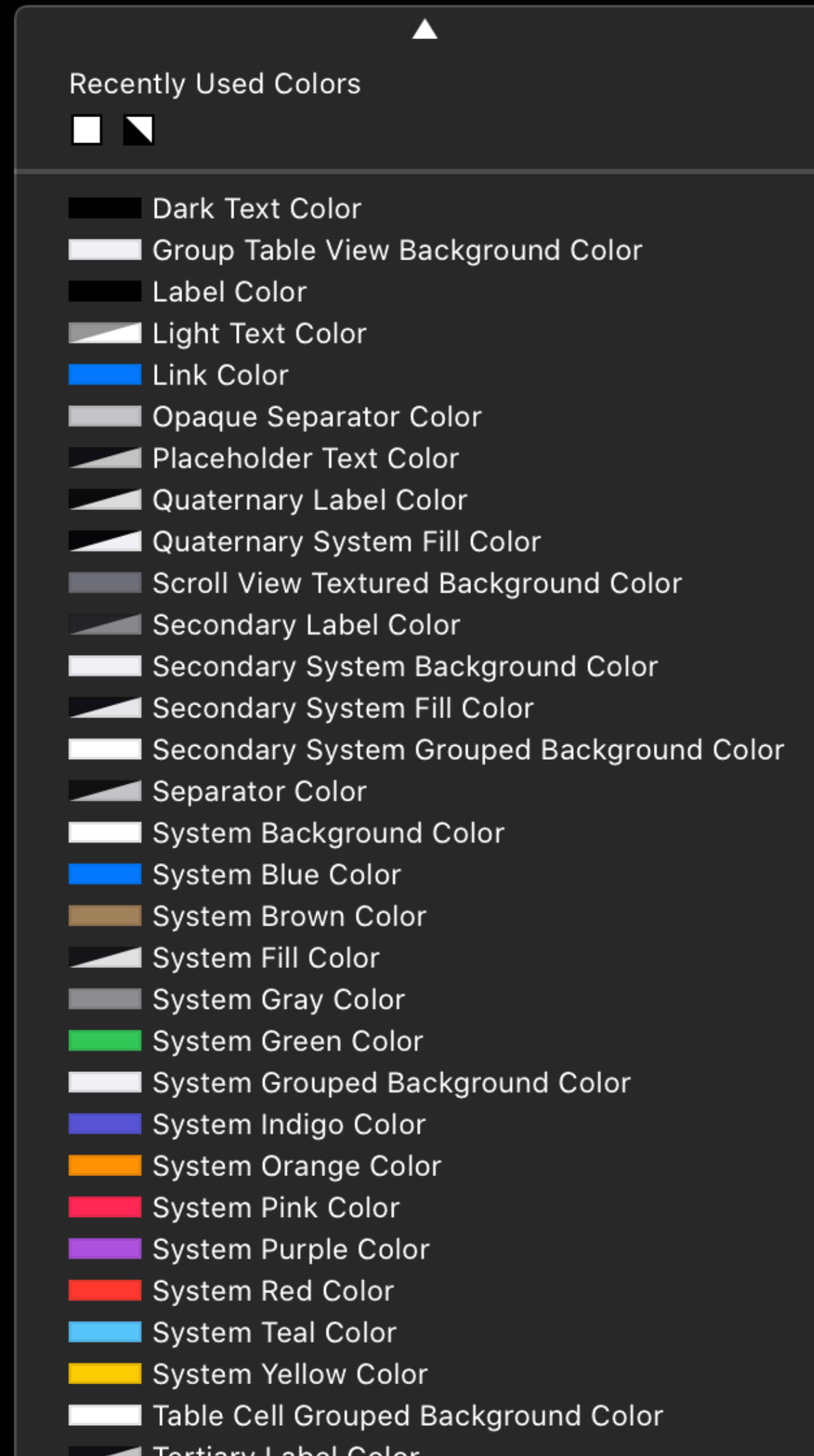
`Color.secondary`

UIKit

```
let blue: UIColor = .systemBlue
```

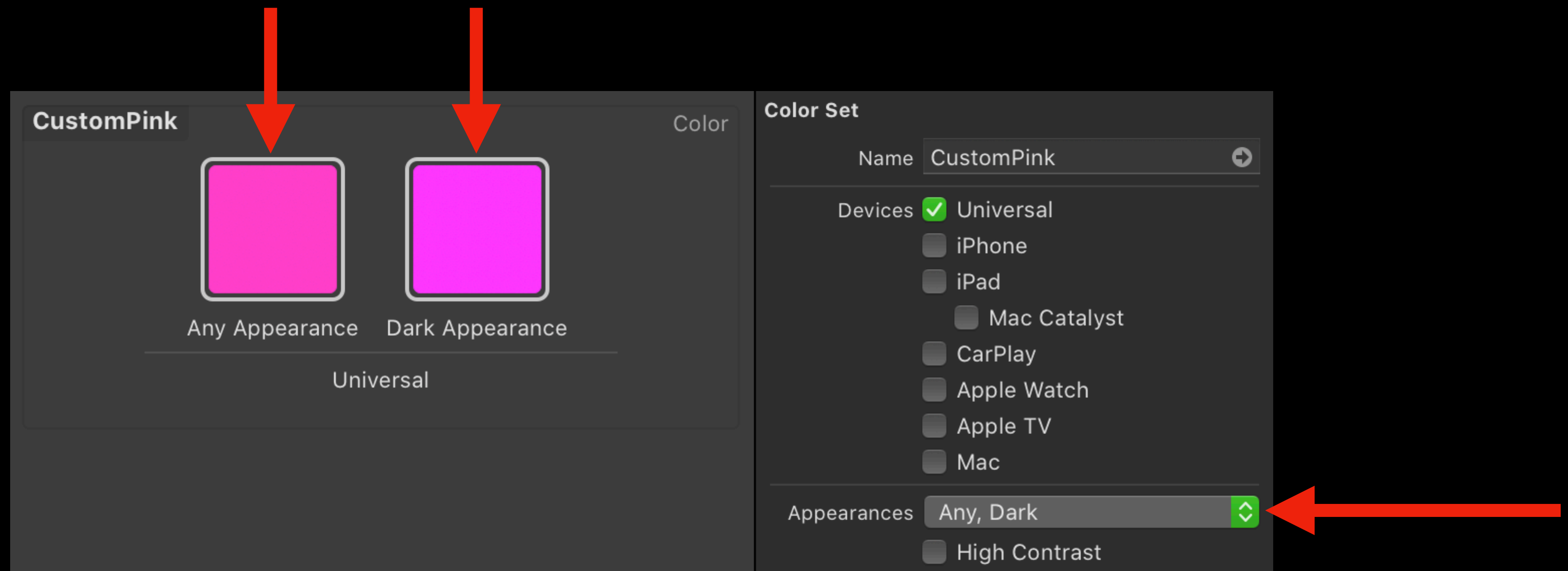
```
let viewBackground: UIColor = .systemBackground
```

Using system colours - interface builder



Custom dynamic colours

Subtle difference depending on
light/dark appearance



Custom dynamic colours

```
let someColour = UIColor { (traitCollection: UITraitCollection) -> UIColor in  
    return traitCollection.userInterfaceStyle == .dark ? .black : .white  
}
```

Trait collection of a view is set before the following methods are called:

UIView

UIViewController

UIPresentationController

`draw()`

`layoutSubviews()`

`viewWillLayoutSubviews()`

`containerViewWillLayoutSubviews()`

`viewDidLayoutSubviews()`

`containerViewDidLayoutSubviews()`

`traitCollectionDidChange()`

`traitCollectionDidChange()`

`traitCollectionDidChange()`

`tintColorDidChange()`

Source:

Implementing Dark Mode on iOS session from WWDC19

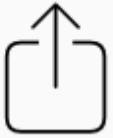
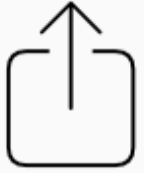
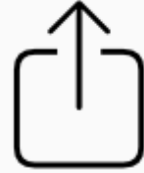
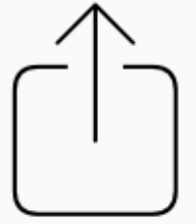
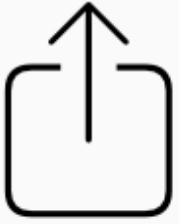
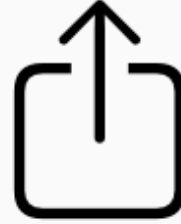
<https://developer.apple.com/videos/play/wwdc2019/214/>

SF Symbols

- Set of symbols/glyphs
- Nine font weights - ultralight to black
- Three scales - small, medium, large
- Can replace a lot of third-party symbols across an app

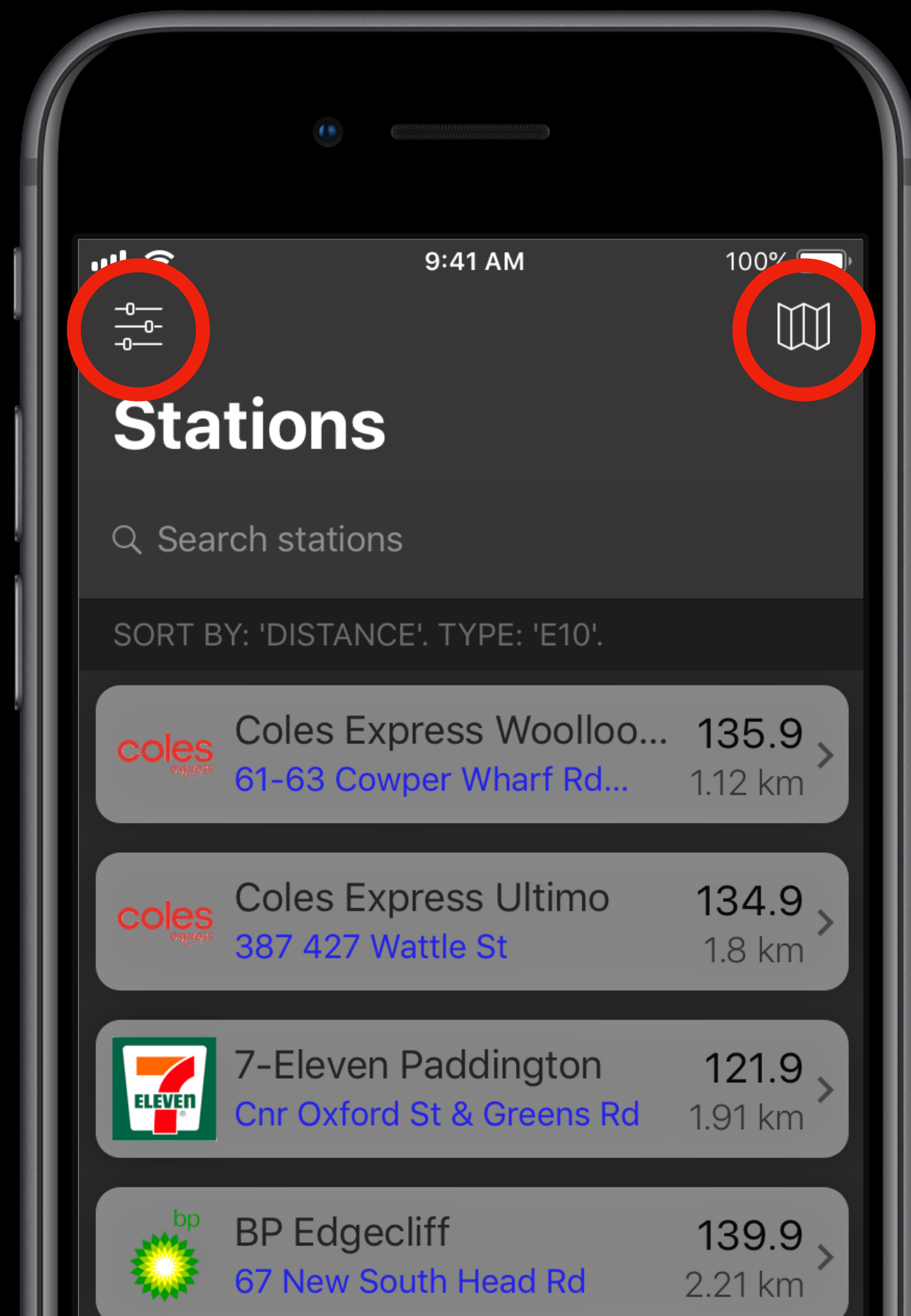
SF Symbols

SF Symbols provides a set of over 1,500 consistent, highly configurable symbols you can use in your app. Apple designed SF Symbols to integrate seamlessly with the San Francisco system font, so the symbols automatically ensure optical vertical alignment with text for all weights and sizes. SF Symbols are available in a wide range of weights and scales to help you create adaptable designs.

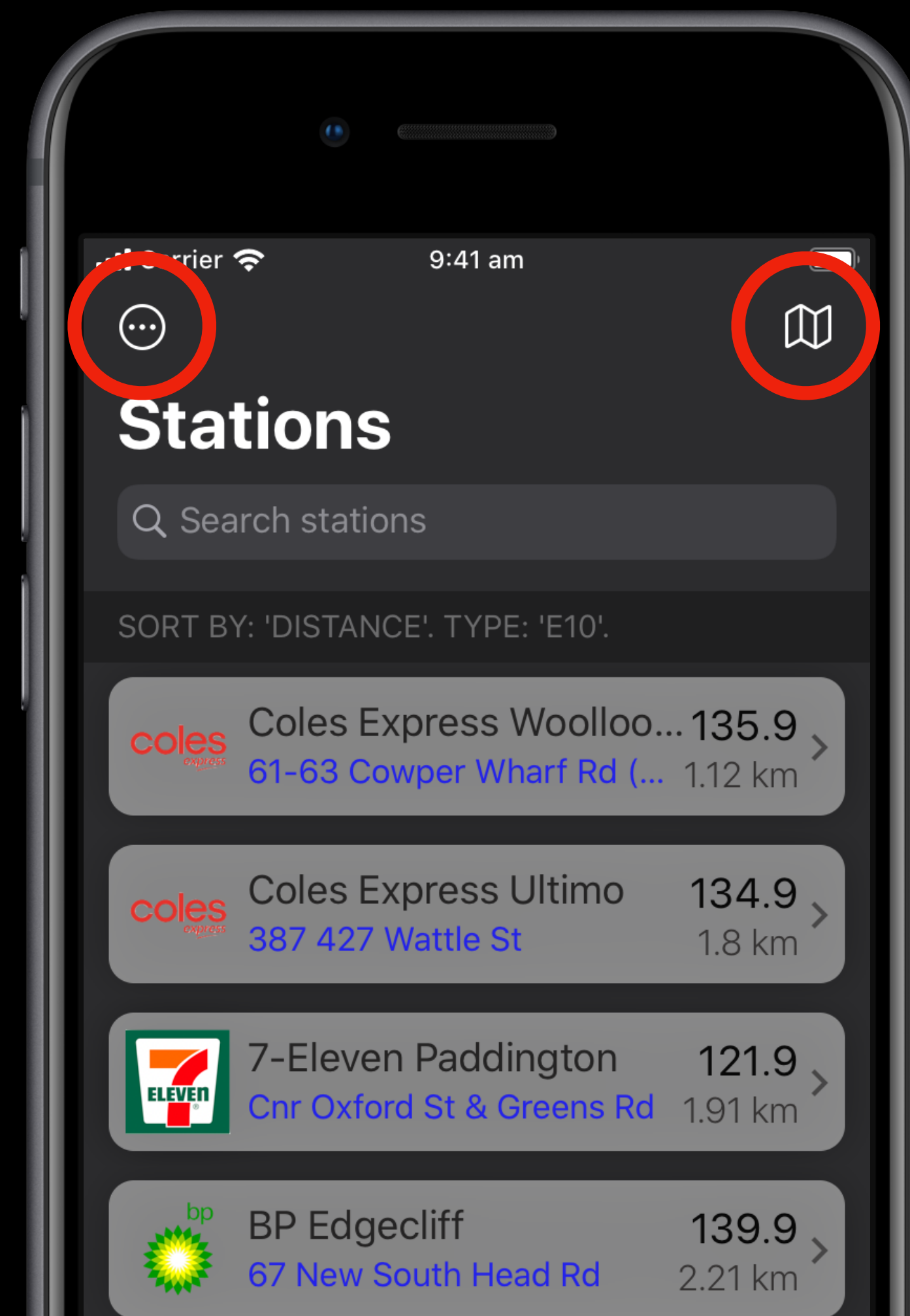
	Ultralight	Thin	Light	Regular	Medium	Semibold	Bold	Heavy	Black
Small									
Medium									
Large									

Using SF Symbols in Petty

iOS 12

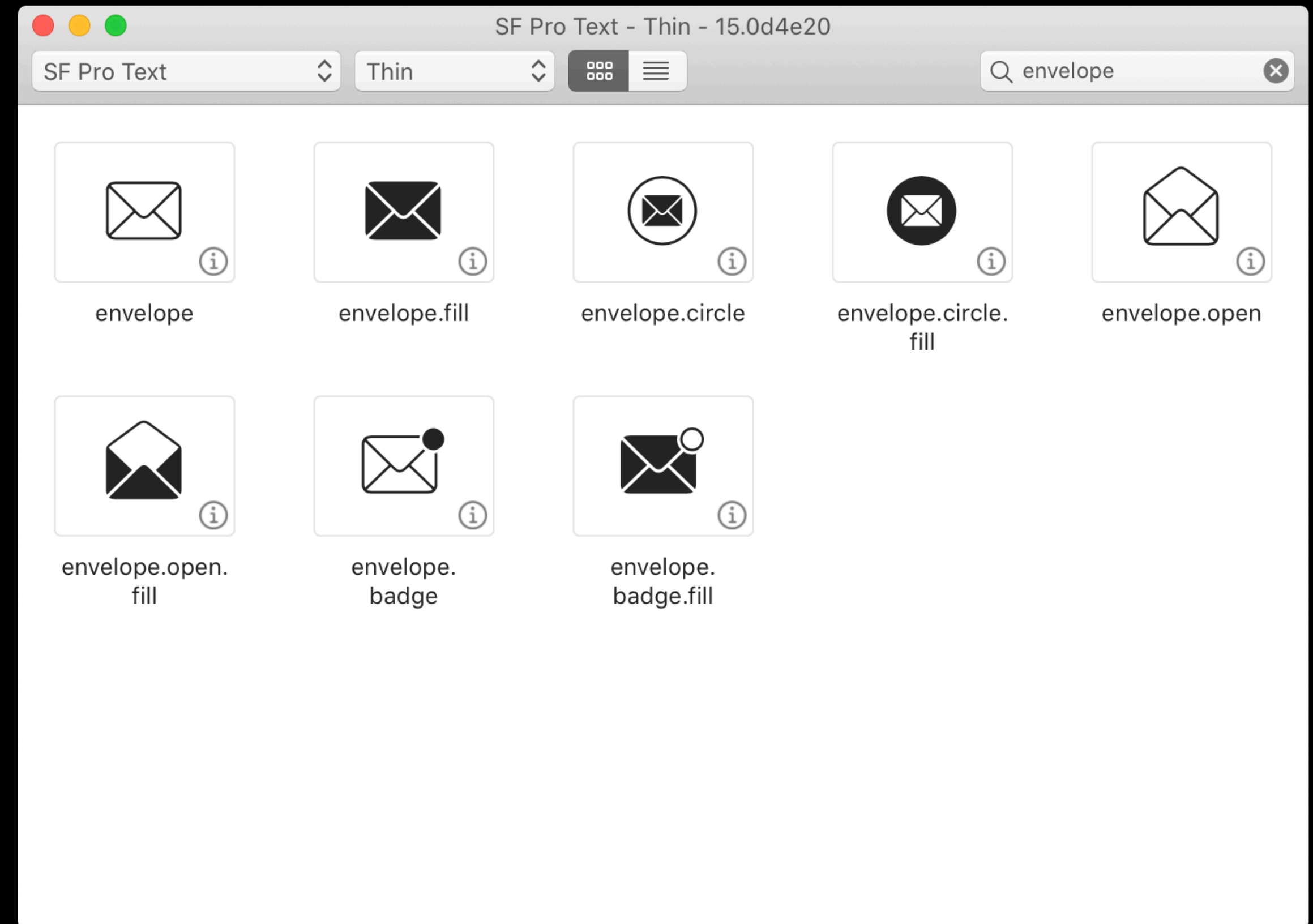
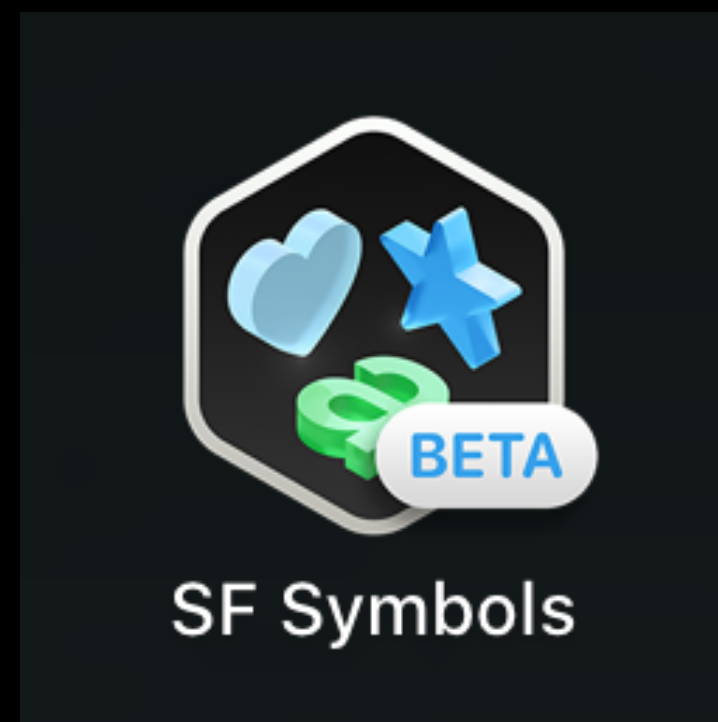


iOS 13



SF Symbols - how?

- SF Symbols macOS app to browse



Code

SwiftUI:

```
Image(systemName: "map").imageScale(.large)
```

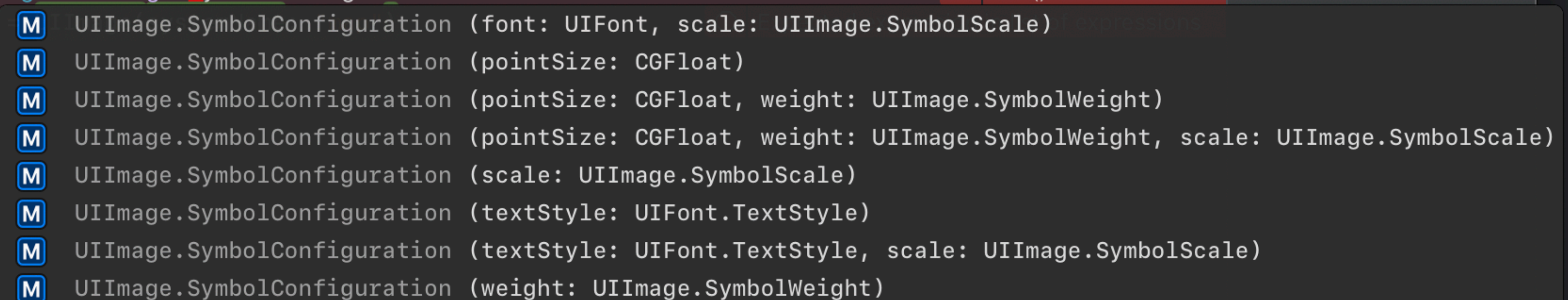
Good ol' UIKit:

```
var mapImage = UIImage(systemName: "map")
```

SF Symbol Configuration in UIKit

UIKit:

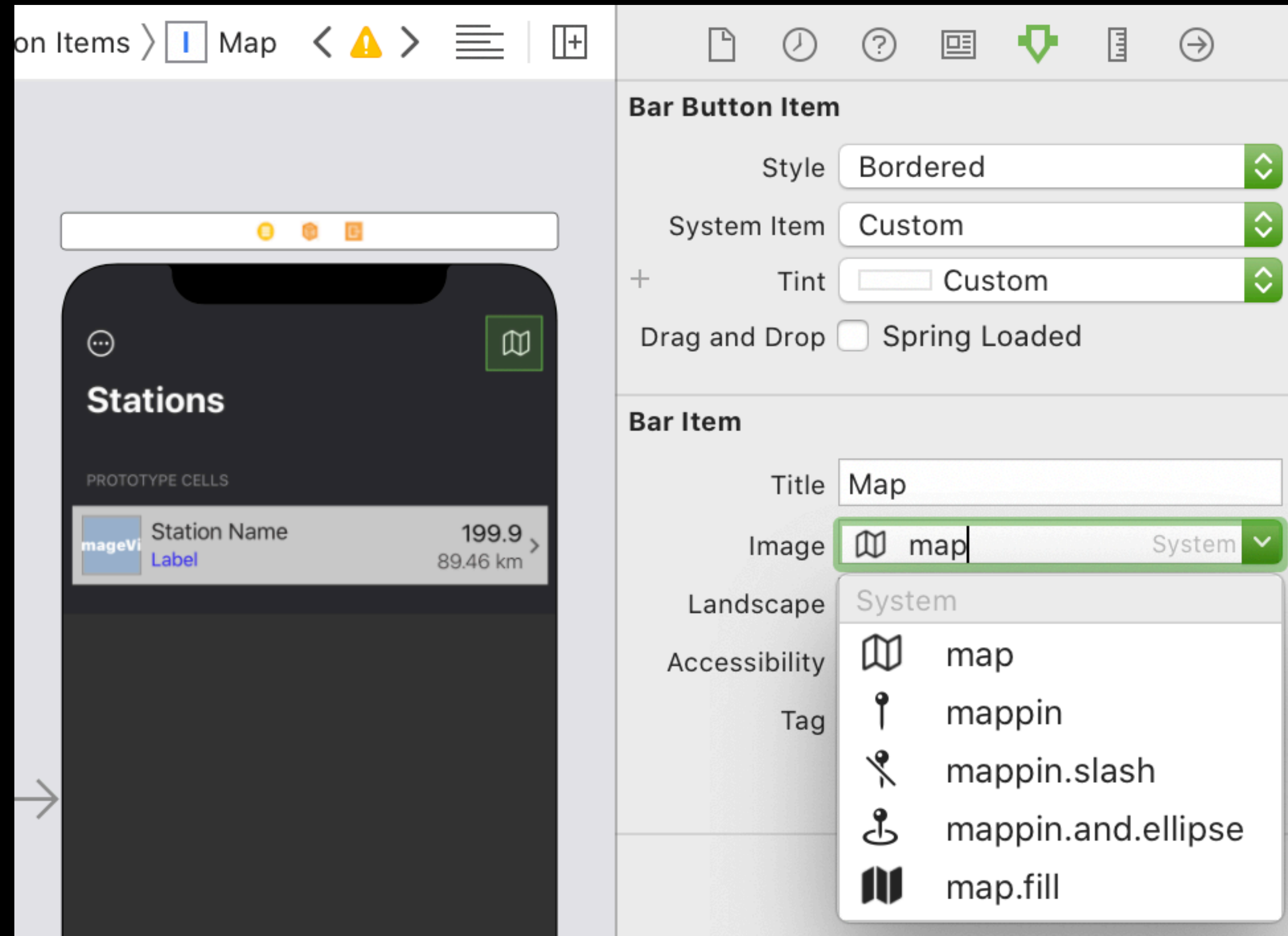
```
let imageConfig = UIImage.SymbolConfiguration(  
var mapImage
```



The image shows an Xcode autocomplete menu for the `UIImage.SymbolConfiguration` class. The menu lists eight different initializer methods, each preceded by a blue square icon containing a white letter 'M'. The methods are: `(font: UIFont, scale: UIImage.SymbolScale)`, `(pointSize: CGFloat)`, `(pointSize: CGFloat, weight: UIImage.SymbolWeight)`, `(pointSize: CGFloat, weight: UIImage.SymbolWeight, scale: UIImage.SymbolScale)`, `(scale: UIImage.SymbolScale)`, `(textStyle: UIFont.TextStyle)`, `(textStyle: UIFont.TextStyle, scale: UIImage.SymbolScale)`, and `(weight: UIImage.SymbolWeight)`. A red error message is visible in the background: `'init()' is unavailable`.

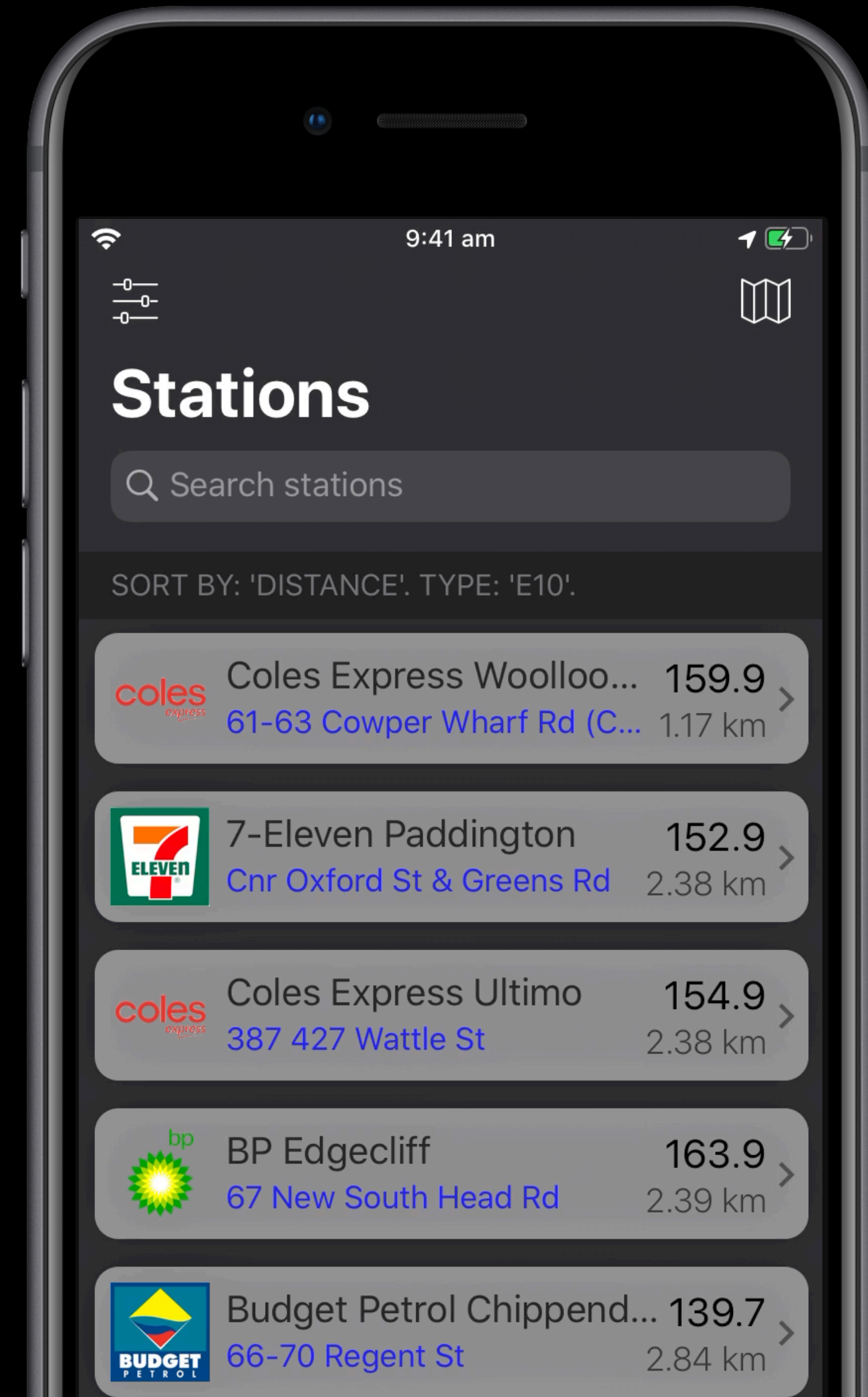
```
let imageConfig = UIImage.SymbolConfiguration(textStyle: .body,  
                                              scale: .large)  
var mapImage = UIImage(systemName: "map",  
                      withConfiguration: imageConfig)
```

Interface Builder



Modal presentations

Swipe-to-dismiss is default



Modal presentations

Full-screen view controller presentation:

```
viewController.modalPresentationStyle = .fullScreen
```

or, to prevent swipe-to-dismiss:

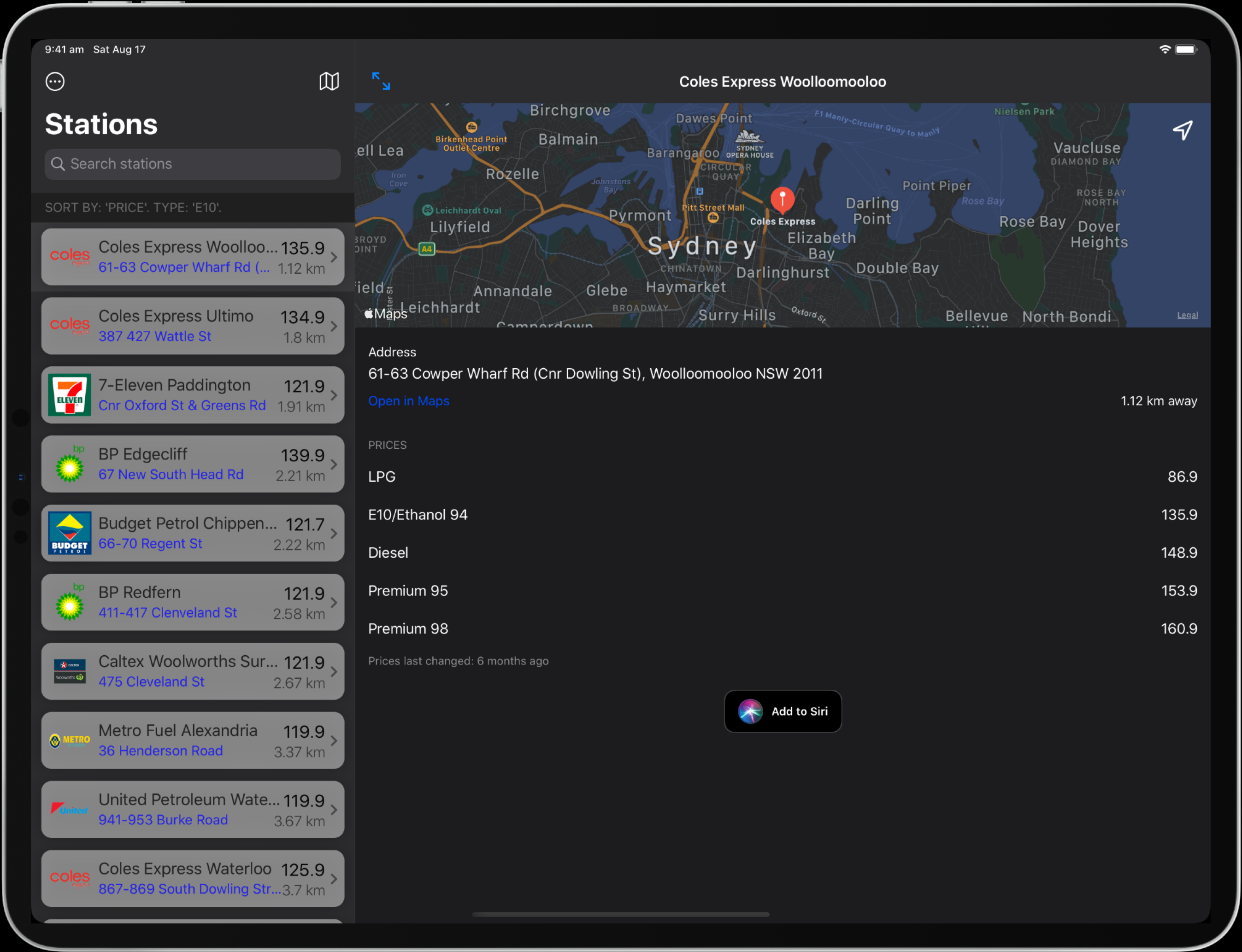
```
viewController.isModalInPresentation = true
```



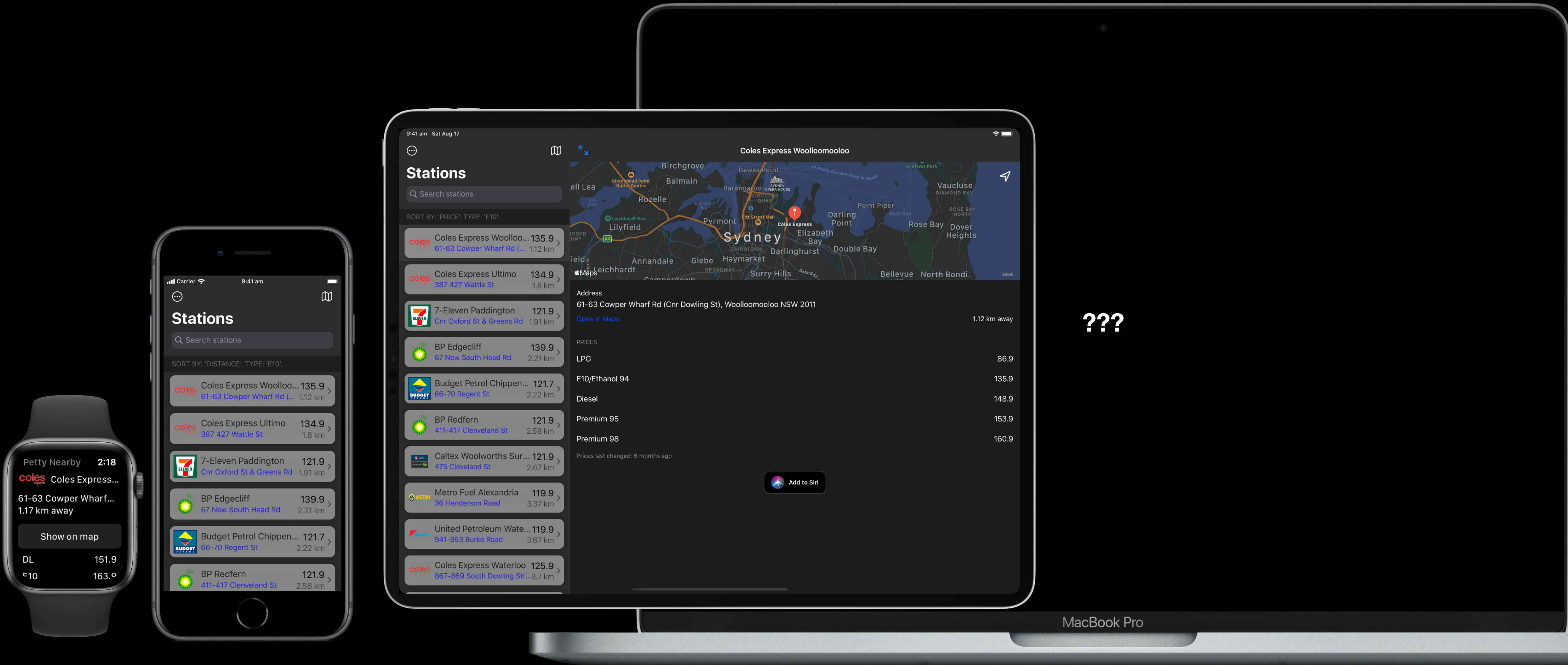
What about iPad?

- Petty has always been iPhone-only
- An iPad target is necessary to bring an app to the Mac using Catalyst
- New navigation structure: UISplitViewController

End result



How about that Mac app?



Tick the box - that's all!

▼ Identity

App Category Utilities

Display Name Petty

Bundle Identifier zachsim.one.Petty

maccatalyst.zachsim.one.Petty on Mac ?

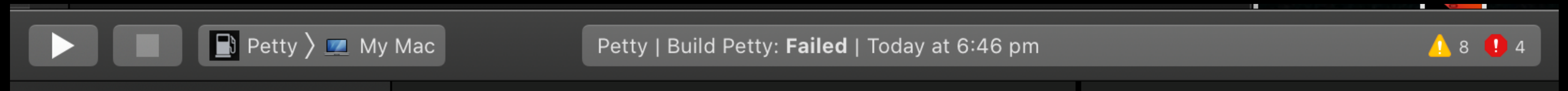
Version 3.0

Build 1

▼ Deployment Info

Target	Device
iOS 13.0	☒ iPhone
	☒ iPad
	☒ Mac (requires macOS 10.15)

Only kidding 🤔



Challenge #1 - Siri Shortcuts

```
#if !targetEnvironment(macCatalyst)
class AddToSiriTableViewCell: UITableViewCell {

    var stationIntent: ShowPricesAtStationIntent?
    private let siriButton =
        INUIAddVoiceShortcutButton(style: .blackOutline)

    ...
}
#endif
```

Challenge #2 - Core Location

App Sandbox

Network

☒

Incoming Connections (Server)

☒

Outgoing Connections (Client)

Hardware

☐

Camera

☐

Audio Input

☐

USB

☐

Printing

☐

Bluetooth

App Data

☐

Contacts

☒

Location

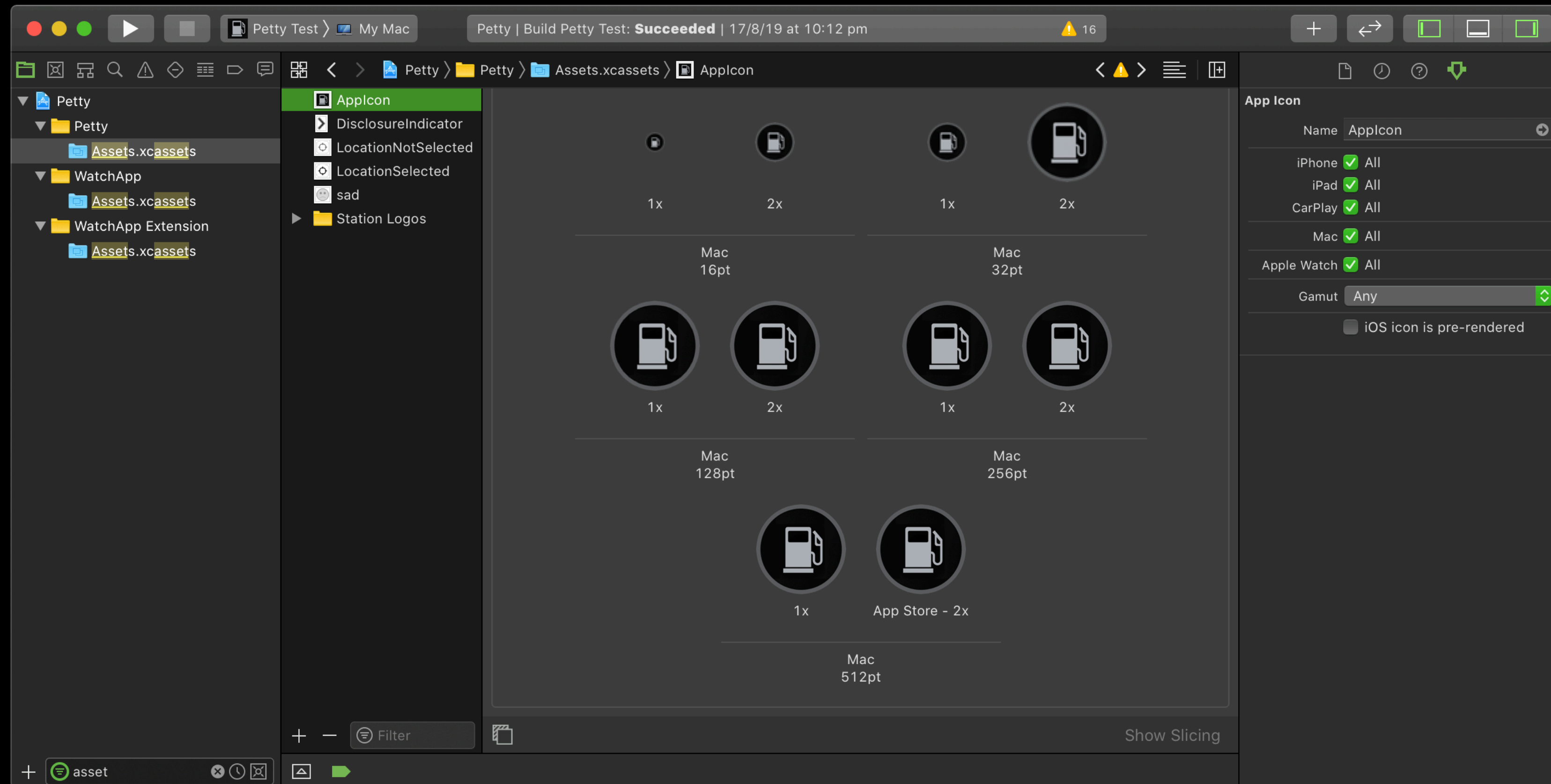
☐

Calendar

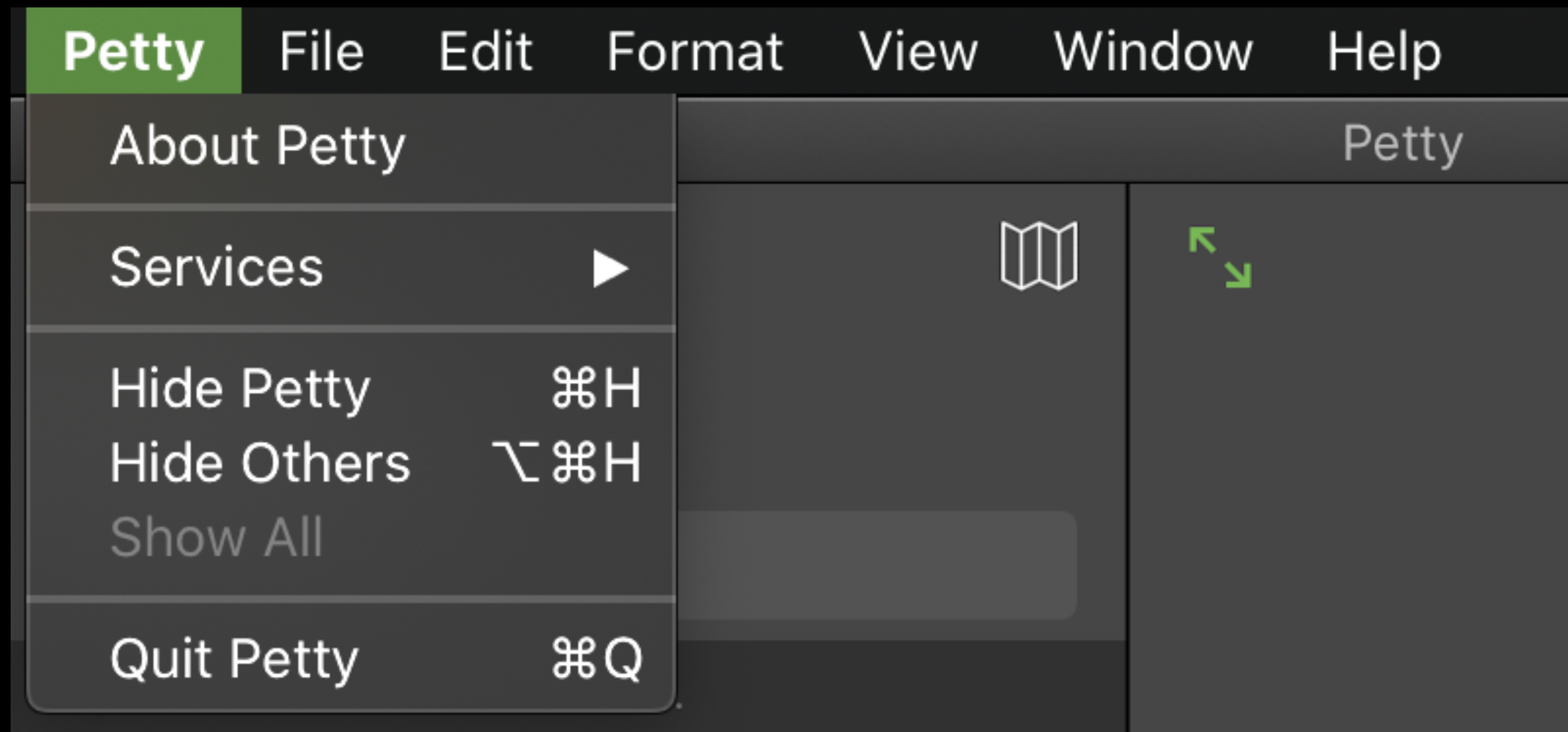
File Access

Type	Permission & Access
User Selected File	None
Downloads Folder	None
Pictures Folder	None
Music Folder	None
Movies Folder	None

Mac app icons



The Menu Bar



Customising the menu bar

```
override func buildMenu(with builder: UIMenuBuilder) {  
    super.buildMenu(with: builder)  
  
    /* Do something */  
}
```

AppDelegate.swift

Customising the menu bar

Removing unwanted menus

```
override func buildMenu(with builder: UIMenuBuilder) {  
    super.buildMenu(with: builder)  
  
    builder.remove(menu: .services)  
    builder.remove(menu: .format)  
    builder.remove(menu: .toolbar)  
}
```



Adding a menu bar item

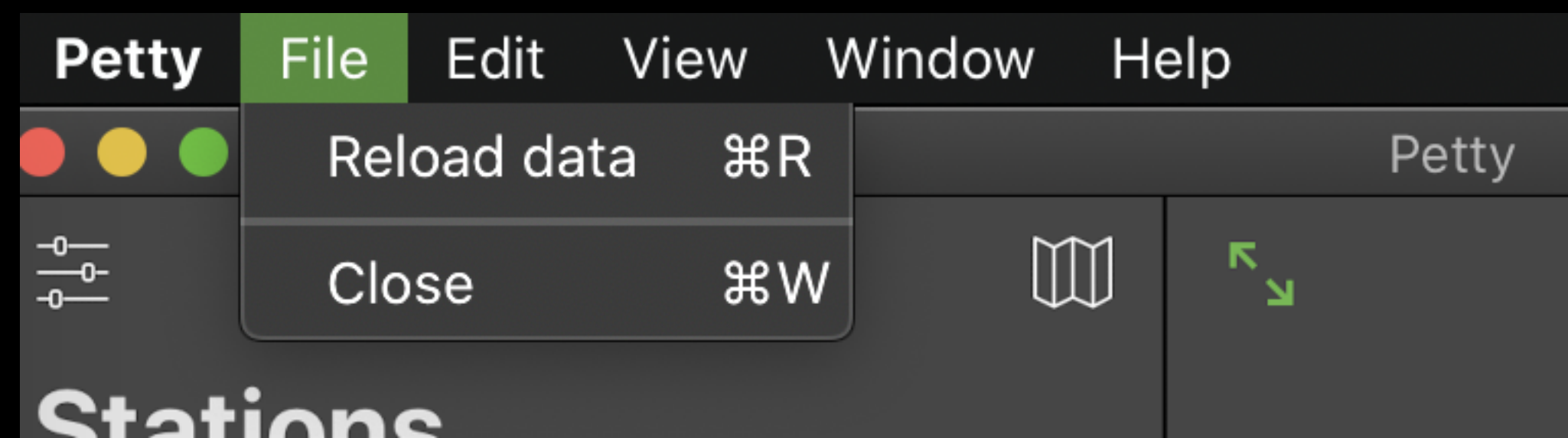
```
let refreshCommand = UIKeyCommand(input: "R",  
                                   modifierFlags: [.command],  
                                   action: #selector(reloadData))  
refreshCommand.title = "Reload data"  
  
let reloadDataMenu = UIMenu(title: "Reload data",  
                             image: nil,  
                             identifier: UIMenu.Identifier("reloadData"),  
                             options: .displayInline,  
                             children: [refreshCommand])  
  
builder.insertChild(reloadDataMenu, atStartOfMenu: .file)
```

Adding a menu bar item

```
let refreshCommand = UIKeyCommand(input: "R",  
                                   modifierFlags: [.command],  
                                   action: #selector(reloadData))  
refreshCommand.title = "Reload data"  
  
let reloadDataMenu = UIMenu(title: "Reload data",  
                             image: nil,  
                             identifier: UIMenu.Identifier("reloadData"),  
                             options: .displayInline,  
                             children: [refreshCommand])  
  
builder.insertChild(reloadDataMenu, atStartOfMenu: .file)
```

Adding a menu bar item

```
let refreshCommand = UIKeyCommand(input: "R",  
                                   modifierFlags: [.command],  
                                   action: #selector(reloadData))  
refreshCommand.title = "Reload data"  
  
let reloadDataMenu = UIMenu(title: "Reload data",  
                             image: nil,  
                             identifier: UIMenu.Identifier("reloadData"),  
                             options: .displayInline,  
                             children: [refreshCommand])  
  
builder.insertChild(reloadDataMenu, atStartOfMenu: .file)
```

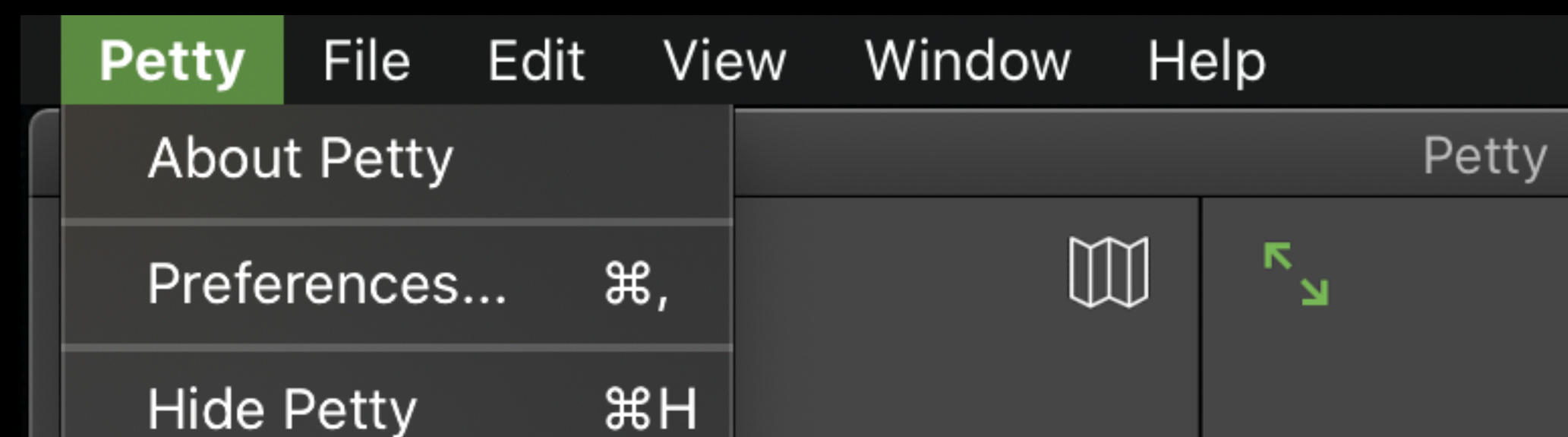


Adding a second menu bar item

```
let preferencesCommand = UIKeyCommand(input: ",",
                                      modifierFlags: [.command],
                                      action: #selector(openPreferences))
preferencesCommand.title = "Preferences..."

let openPreferences = UIMenu(title: "Preferences...",
                             image: nil,
                             identifier: UIMenu.Identifier("openPreferences"),
                             options: .displayInline,
                             children: [preferencesCommand])

builder.insertSibling(openPreferences, afterMenu: .about)
```



Distributing a Catalyst app

- Archive in Xcode
- Automatic code signing will register the bundle ID automatically (and prefix it with `macCatalyst.\*`)
- Create new app record in App Store Connect
- Possible to distribute outside of Mac App Store, can still get app notarised

Learn more:

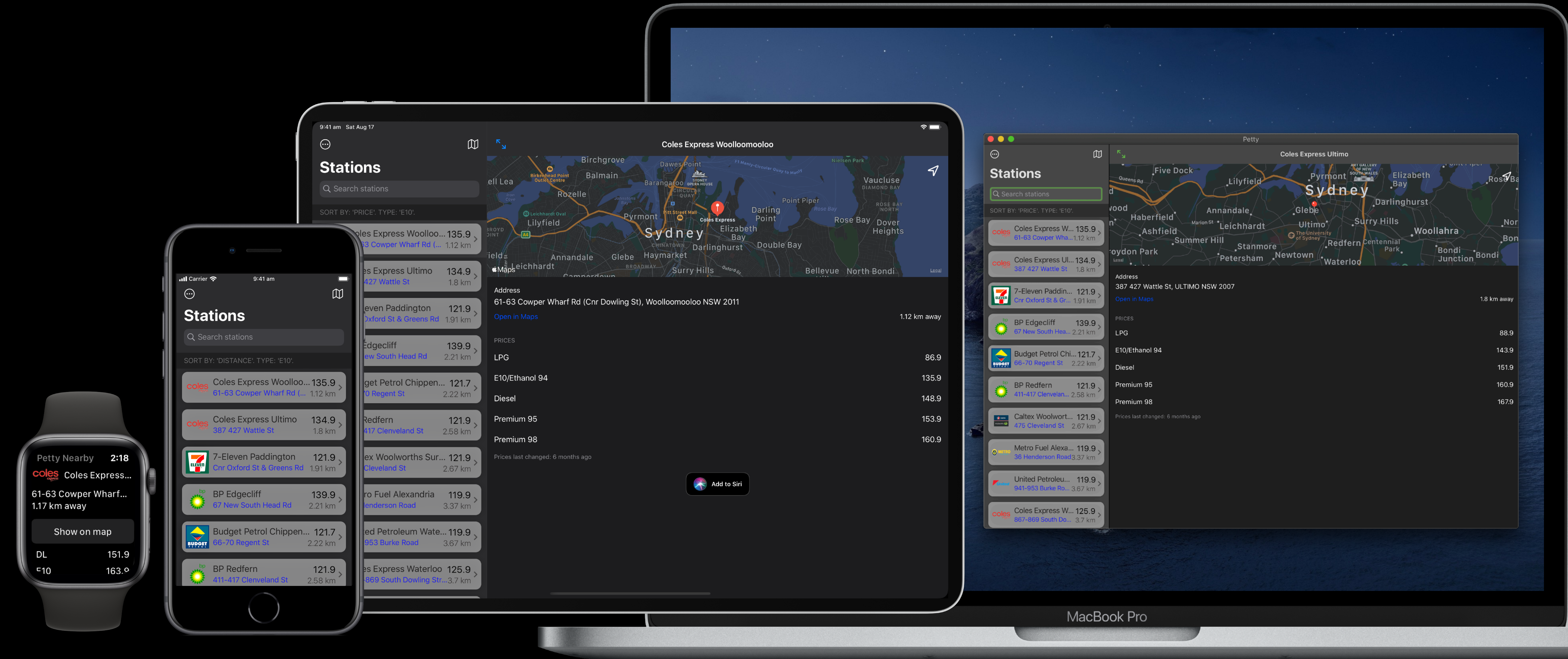
Taking iPad Apps for Mac to the Next Level

<https://developer.apple.com/videos/play/wwdc2019/235/>

Distribution notes

- No TestFlight for macOS apps
- A notarised app cannot use App Store-based features such as GameKit and SceneKit
- App Thinning isn't used on macOS - the bundle contains all the resources.
- Many ways to install a Mac app. Should use receipt validation if legitimacy of install needs to be checked.
- In-app purchases and subscriptions will need to be re-created for the Mac app, and are not shared with iOS. Tracking purchase history requires custom server-side checking.

The lineup



What we covered in this talk

- Making the watch app independent
- Updating for iOS 13 design changes: Dark mode, dynamic colours, SF Symbols, changes to modal presentations
- Mac Catalyst: Conditional code compilation, entitlements, app icon, the menu bar, distribution

There's still so much more!

What wasn't covered

iOS 13:

- Siri Shortcuts/Intents API changes

Catalyst:

- Hover states
- Context menus
- Touch Bar API's
- Navigation bar and title



It's time to get developing!

Thank you. 🙌